

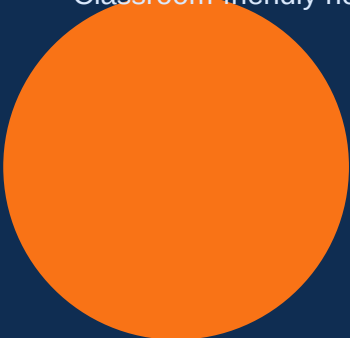


APACHE SPARK

Complete Notes

RDDs | DataFrames | DAG | Spark SQL | PySpark | Performance

Classroom-friendly notes with diagrams, code examples, exercises, MCQs and interview Q&A



Prepared for ItTechGenie Training

Table of Contents

- 1. What is Apache Spark?
- 2. Why Spark? Spark vs Hadoop MapReduce
- 3. Spark Architecture
- 4. Spark Core Concepts
- 5. RDDs: creation, transformations and actions
- 6. DataFrames, Spark SQL and Datasets
- 7. DAG and lazy evaluation
- 8. PySpark practical examples
- 9. Spark ecosystem
- 10. Performance tuning and best practices
- 11. Troubleshooting
- 12. Hands-on exercises
- 13. MCQs and interview Q&A;

How to use these notes

Read Section 1 to 4 for fundamentals. Practice Section 8 commands in Spark Shell or PySpark. Use Section 10 and 11 during project debugging. Use Section 13 for interview preparation.

1. What is Apache Spark?

Apache Spark is a distributed data processing engine used for large-scale batch processing, interactive analytics, streaming, machine learning and graph processing. It can run on a single laptop for practice or on a cluster with many machines for production workloads.

Spark's main strength is that it keeps intermediate data in memory whenever possible. This makes many workloads faster than traditional disk-heavy batch engines, especially iterative workloads such as machine learning and repeated analytics.

Simple definition

Spark is like a smart factory manager for big data. It divides a large job into small tasks, sends those tasks to multiple workers, and combines the results.

Where Spark is used

- ETL and data engineering pipelines: clean, transform and load large datasets.
- Analytics: calculate reports, KPIs, aggregations and joins across large data.
- Streaming: process events from Kafka, files, sockets or cloud event systems.
- Machine learning: feature engineering, model training and scoring using MLlib or external libraries.
- Data lakehouse processing: read and write formats like Parquet, ORC, Delta and Iceberg depending on platform support.

Memory aid

Remember: S-P-A-R-K

S - Scalable, P - Parallel, A - Analytics, R - Resilient, K - Keeps data in memory when useful.

2. Why Spark? Spark vs Hadoop MapReduce

Hadoop MapReduce is reliable for batch jobs, but every Map and Reduce step usually writes intermediate results to disk. Spark improves productivity and speed by building a DAG of operations and reusing memory across steps when possible.

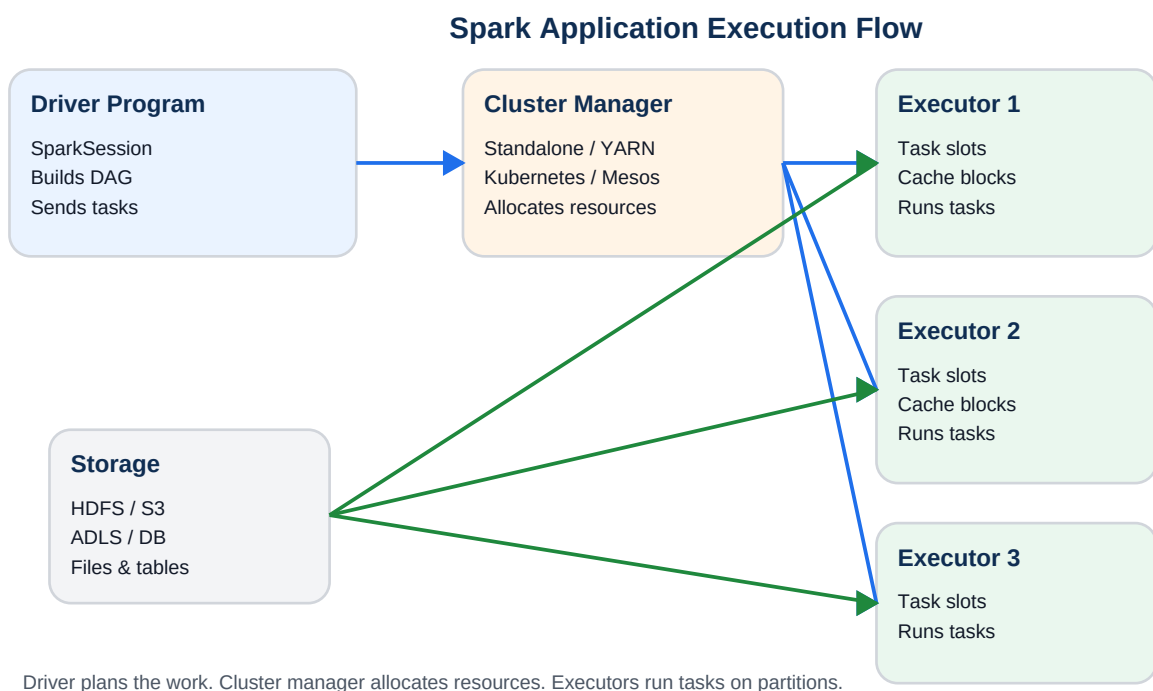
Area	Hadoop MapReduce	Apache Spark
Execution model	Map phase followed by Reduce phase.	DAG-based execution with multiple stages and tasks.
Speed	Good for batch, but disk I/O is heavy.	Often faster for iterative and interactive workloads because of memory usage.
Ease of use	More boilerplate code.	Higher-level APIs in Scala, Java, Python, SQL and R.
Streaming	Not designed for real-time streaming.	Structured Streaming supports continuous processing style.

Area	Hadoop MapReduce	Apache Spark
Use case	Classic batch processing.	Batch, SQL, streaming, ML and graph processing.

When MapReduce may still be enough

For very simple batch jobs that run rarely and where existing MapReduce code is stable, MapReduce can still be acceptable. For modern analytics, Spark is usually preferred because development is faster and the execution engine is more flexible.

3. Spark Architecture



Main components

Component	Meaning	Responsibility
Driver	The main process of the Spark application.	Creates SparkSession/SparkContext, builds the DAG, schedules tasks and collects results.
Cluster Manager	Resource manager for the cluster.	Allocates CPU and memory resources. Examples: Standalone, YARN, Kubernetes.
Executor	Worker-side JVM process.	Runs tasks, stores cached data and reports status to the driver.
Task	Smallest unit of work.	Processes one partition of data.
Partition	A slice of a distributed dataset.	Enables parallel processing across executor cores.

Flow of execution

- 1 User submits a Spark application.

- 2 Driver starts and requests resources from the cluster manager.
- 3 Cluster manager starts executor processes on worker nodes.
- 4 Driver converts transformations into a DAG and splits it into stages.
- 5 Executors run tasks on partitions and return status/results to the driver.
- 6 Output is displayed, saved to storage or written to a target table/system.

4. Spark Core Concepts

SparkSession

SparkSession is the entry point for Spark programming in modern Spark applications. It provides access to DataFrame, SQL, catalog and configuration features.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder      .appName("SparkNotesDemo")      .master("local[*]")      .getOrCreate()
```

RDD, DataFrame and Dataset

API	Description	When to use
RDD	Low-level distributed collection of objects.	When you need fine-grained control or are learning Spark fundamentals.
DataFrame	Distributed table with named columns and schema.	Preferred API for most data engineering and analytics work.
Dataset	Typed distributed collection in Scala/Java.	Useful when compile-time type safety is required in Scala/Java.

Transformations and actions

Transformations create a new logical dataset from an existing one. Actions trigger execution and return a result or write output.

Category	Examples	Execution behavior
Narrow transformations	map, filter, select, withColumn	Data can be processed partition by partition; usually no shuffle.
Wide transformations	groupBy, reduceByKey, join, distinct	Data may move across the network; creates shuffle and stage boundary.
Actions	count, collect, show, take, save	Triggers the actual job execution.

5. RDDs: Creation, Transformations and Actions

RDD stands for Resilient Distributed Dataset. It is immutable, partitioned and fault tolerant. RDDs are important for understanding Spark internals even though DataFrames are preferred in most production projects.

Different ways to create RDD

Method	Example	Explanation
Parallelized collection	<code>sc.parallelize([1, 2, 3])</code>	Create RDD from local Python/Scala collection.
External dataset	<code>sc.textFile('path')</code>	Create RDD from HDFS, local file, S3 or similar storage.
Existing RDD	<code>rdd.filter(...)</code>	Create a new RDD by transforming another RDD.

RDD examples in PySpark

```
# Create RDD using parallelized collection
numbers = sc.parallelize([10, 20, 30, 40, 50])

# Transformation: filter
high = numbers.filter(lambda x: x >= 30)

# Action: collect
print(high.collect())      # [30, 40, 50]
print(high.count())        # 3
print(high.take(2))        # [30, 40]
```

Read file and create RDD

```
lines = sc.textFile("/user/training/input/sales.txt")
print(lines.count())
print(lines.take(5))
```

Word count example

```
text = sc.textFile("/user/training/input/words.txt")

counts = (text
    .flatMap(lambda line: line.split())
    .map(lambda word: (word.lower(), 1))
    .reduceByKey(lambda a, b: a + b))

counts.saveAsTextFile("/user/training/output/wordcount")
```

Trainer note

Use RDDs to teach partitions, lazy evaluation, transformations, actions and fault tolerance. Use DataFrames for real project work.

6. DataFrames, Spark SQL and Datasets

A DataFrame is a distributed table with rows and columns. It has a schema, supports SQL-like operations and is optimized by Spark's Catalyst optimizer.

Create DataFrame from Python list

```
data = [(1, "Asha", "DE", 60000),
        (2, "Ravi", "QA", 45000),
        (3, "Meena", "DE", 75000)]

columns = ["id", "name", "dept", "salary"]
df = spark.createDataFrame(data, columns)
df.show()
df.printSchema()
```

Common DataFrame operations

```
from pyspark.sql.functions import col, avg, count

# Select columns
df.select("name", "salary").show()

# Filter rows
df.filter(col("salary") > 50000).show()

# Add/modify column
df.withColumn("bonus", col("salary") * 0.10).show()

# Group and aggregate
df.groupBy("dept").agg(count("*").alias("emp_count"),
                       avg("salary").alias("avg_salary")).show()
```

Spark SQL

```
df.createOrReplaceTempView("employees")

result = spark.sql("""
SELECT dept, COUNT(*) AS emp_count, AVG(salary) AS avg_salary
FROM employees
GROUP BY dept
ORDER BY avg_salary DESC
""")

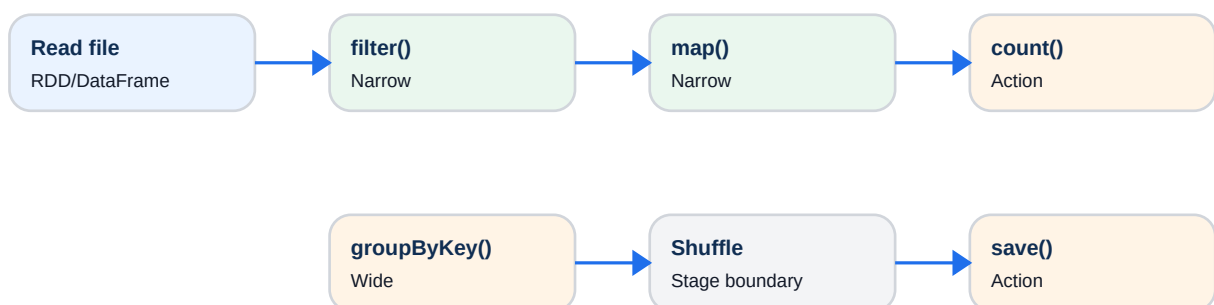
result.show()
```

Why DataFrames are preferred

- They are easier to read and maintain than low-level RDD code.
- They use schema information for optimization.
- They support SQL, joins, aggregations, window functions and file formats.
- They integrate well with lakehouse platforms and BI workflows.

7. DAG and Lazy Evaluation

Lazy Evaluation and DAG



Transformations create a logical plan. Actions trigger execution. Wide transformations create shuffle and stage boundaries.

Spark does not execute transformations immediately. It records them as a lineage graph. When an action is called, Spark analyzes the lineage, creates a DAG, splits the DAG into stages, and executes tasks.

Important terms

Term	Meaning
Lineage	History of transformations used to create an RDD/DataFrame.
DAG	Directed Acyclic Graph of operations. It represents execution dependency.
Job	Work triggered by an action.
Stage	Group of tasks separated by shuffle boundaries.
Task	Small unit of work sent to executor for one partition.
Shuffle	Movement of data across partitions/nodes, often caused by joins and aggregations.

Example: action triggers execution

```
df2 = df.filter(col("salary") > 50000) # transformation: no execution yet
df3 = df2.select("name", "salary")      # transformation: no execution yet

# show() is an action, so Spark now creates jobs/stages/tasks
df3.show()
```

Memory aid

Transformations are planning. Actions are execution.

8. PySpark Practical Examples

Read CSV

```
sales = spark.read.option("header", True).option("inferSchema", True).csv("/data/sales.csv")

sales.show(5)
sales.printSchema()
```

Clean and transform data

```
from pyspark.sql.functions import col, trim, upper, to_date

clean_sales = (sales
    .withColumn("customer_name", trim(col("customer_name")))
    .withColumn("country", upper(col("country")))
    .withColumn("order_date", to_date(col("order_date"), "yyyy-MM-dd"))
    .filter(col("amount") > 0))

clean_sales.show(10)
```

Aggregation

```
from pyspark.sql.functions import sum as spark_sum, count

summary = (clean_sales
    .groupBy("country")
    .agg(count("*").alias("orders"),
        spark_sum("amount").alias("total_sales"))
    .orderBy(col("total_sales").desc()))

summary.show()
```

Join example

```
customers = spark.read.option("header", True).csv("/data/customers.csv")
```



```
sales_with_customer = clean_sales.join(customers, on="customer_id", how="inner")
sales_with_customer.select("order_id", "customer_name", "amount").show()
```

Write output

```
summary.write .mode("overwrite") .format("parquet") .save("/data/output/country_sales")
```

9. Spark Ecosystem

Component	Purpose	Example use case
Spark Core	Base execution engine, scheduling, memory management and RDD API.	Distributed word count, custom transformations.
Spark SQL	DataFrame and SQL processing.	Data warehouse style analytics and ETL.
Structured Streaming	Streaming API built on DataFrames.	Process Kafka events and write near real-time dashboards.
MLlib	Machine learning library.	Classification, regression, clustering and feature engineering.
GraphX	Graph computation API in Scala.	PageRank, social network analysis.
SparkR	R interface for Spark.	R users working with distributed DataFrames.

Structured Streaming mini example

```
stream_df = spark.readStream .format("socket") .option("host", "localhost") .option("port", 9999)

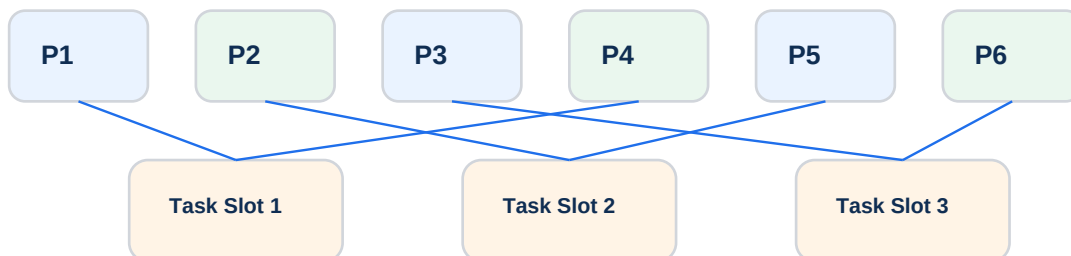
words = stream_df.selectExpr("explode(split(value, ' ')) as word")
counts = words.groupBy("word").count()

query = counts.writeStream .outputMode("complete") .format("console") .start()

query.awaitTermination()
```

10. Performance Tuning and Best Practices

Partitions and Parallelism



One task usually processes one partition. More partitions can increase parallelism, but too many small partitions add overhead.

Caching and persistence

Use caching when the same DataFrame/RDD is reused multiple times. Do not cache everything; cached data uses executor memory.

```
filtered = df.filter(col("salary") > 50000).cache()
filtered.count()      # materializes cache
filtered.groupBy("dept").count().show()
filtered.unpersist()  # release memory when done
```

Partitioning

- A partition is a unit of parallel processing.
- Too few partitions can underuse CPU cores.
- Too many tiny partitions can increase scheduling overhead.
- Use `repartition()` to increase or reshuffle partitions; use `coalesce()` to reduce partitions with less shuffle.

Shuffle optimization

- Avoid unnecessary wide transformations.
- Filter early to reduce data before joins and `groupBy`.
- Use broadcast join when joining a large table with a small lookup table.
- Prefer `reduceByKey` over `groupByKey` for RDD aggregations.
- Use column pruning: select only required columns.

Broadcast join example

```
from pyspark.sql.functions import broadcast

result = large_sales.join(broadcast(small_country_lookup), "country_code")
```

File format best practices

Format	Use	Notes
CSV	Simple exchange and initial learning.	Slow for large analytics; no rich schema; avoid for production fact tables.
JSON	Semi-structured events.	Flexible but can be expensive to parse.
Parquet	Analytics and data lake storage.	Columnar, compressed, schema-aware and commonly preferred.
ORC	Hive-heavy ecosystems.	Columnar format with good compression.

11. Troubleshooting Common Spark Issues

Problem	Likely reason	Fix
Job is slow	Large shuffle, skewed data, too many small files or insufficient resources.	Check Spark UI, reduce shuffle, filter early, tune partitions and use proper file format.
OutOfMemoryError	Large <code>collect()</code> , huge join, too much cache or poor partitioning.	Avoid <code>collect()</code> on large data, increase partitions, unpersist cache, tune executor memory.
Many tiny output files	Too many partitions at write time.	Use <code>coalesce()</code> or <code>repartition()</code> before write based on target size.

Problem	Likely reason	Fix
Task failed repeatedly	Bad data, executor crash, network issue or code error.	Read executor logs, isolate bad records, validate schema and handle nulls.
Driver memory issue	Using collect(), toPandas() or large result returned to driver.	Write to storage instead of collecting; limit rows for debugging.

Spark UI checklist

- Jobs tab: how many jobs were triggered?
- Stages tab: which stage is slow or failed?
- Tasks tab: are some tasks much slower than others? This may indicate skew.
- SQL tab: review the physical plan and expensive joins/aggregations.
- Executors tab: check memory, GC time, failed tasks and storage usage.

12. Hands-on Exercises

Exercise 1: RDD basics

Create an RDD from numbers 1 to 20. Filter even numbers, square them and display the result.

```
numbers = sc.parallelize(range(1, 21))
result = numbers.filter(lambda x: x % 2 == 0).map(lambda x: x * x)
print(result.collect())
```

Exercise 2: DataFrame filtering

Create an employee DataFrame and show employees whose salary is greater than 50000.

```
employees = [(1, "Asha", 60000), (2, "Ravi", 45000), (3, "Meena", 75000)]
df = spark.createDataFrame(employees, ["id", "name", "salary"])
df.filter(col("salary") > 50000).show()
```

Exercise 3: Aggregation

Using a sales DataFrame with columns country and amount, calculate total sales by country.

```
sales.groupBy("country").agg(spark_sum("amount").alias("total_sales")).show()
```

Exercise 4: Mini project

Build a small ETL pipeline: read sales CSV, clean customer names, remove invalid amount rows, aggregate by country, and write output as Parquet.

```
from pyspark.sql.functions import col, trim, sum as spark_sum

sales = spark.read.option("header", True).option("inferSchema", True).csv("sales.csv")

output = (sales
    .withColumn("customer_name", trim(col("customer_name")))
    .filter(col("amount") > 0)
    .groupBy("country")
    .agg(spark_sum("amount").alias("total_sales")))

output.write.mode("overwrite").parquet("output/country_sales")
```

13. MCQs for Practice

Choose the best answer. Answer key is provided after the questions.

1. What is the main entry point for modern PySpark applications?

- A. SparkSession
- B. HadoopSession
- C. Mapper
- D. Reducer

2. Which Spark operation triggers execution?

- A. filter
- B. map
- C. select
- D. count

3. What does RDD stand for?

- A. Relational Data Design
- B. Resilient Distributed Dataset
- C. Runtime Data Driver
- D. Recursive Data Dataset

4. Which transformation usually causes shuffle?

- A. map
- B. filter
- C. groupBy
- D. select

5. Which API is generally preferred for most Spark ETL work?

- A. DataFrame
- B. Thread
- C. Servlet
- D. Mapper class

6. What is a partition?

- A. A Spark library
- B. A slice of distributed data
- C. A SQL table only
- D. A driver log

7. Which method is used to cache a DataFrame?

- A. save()
- B. cache()

- C. trigger()
- D. compile()

8. What does lazy evaluation mean?

- A. Spark never executes jobs
- B. Transformations are planned and actions trigger execution
- C. Driver stops executors
- D. Files are deleted after execution

9. Which file format is commonly preferred for analytics in data lakes?

- A. Parquet
- B. TXT
- C. BMP
- D. DOCX

10. Which component runs tasks?

- A. Executor
- B. Browser
- C. SQL Server agent
- D. Notebook only

Answer Key

1-A, 2-D, 3-B, 4-C, 5-A, 6-B, 7-B, 8-B, 9-A, 10-A

14. Interview Questions and Answers

Q1. What is Apache Spark?

Answer: Apache Spark is a distributed processing engine for big data workloads such as batch ETL, SQL analytics, streaming and machine learning.

Q2. What is the difference between RDD and DataFrame?

Answer: RDD is a low-level distributed collection. DataFrame is a higher-level table-like API with schema and optimizer support.

Q3. What is lazy evaluation?

Answer: Spark records transformations without executing them immediately. Execution starts only when an action is called.

Q4. What is DAG in Spark?

Answer: DAG stands for Directed Acyclic Graph. It represents the dependency graph of operations required to compute the final result.

Q5. What is a Spark job?

Answer: A job is the execution triggered by an action such as count, collect, show or save.

Q6. What is a stage?

Answer: A stage is a set of tasks that can run together without shuffle. Wide transformations create stage boundaries.

Q7. What is a task?

Answer: A task is the smallest unit of work sent to an executor, usually operating on one partition.

Q8. What is shuffle?

Answer: Shuffle is the movement of data across partitions/nodes, commonly caused by join, groupBy, distinct and repartition.

Q9. Why is collect() dangerous?

Answer: collect() brings all results to the driver. If the result is large, the driver can run out of memory.

Q10. When should cache() be used?

Answer: Use cache when the same DataFrame/RDD is reused multiple times and recomputation is expensive.

Q11. What is repartition()?

Answer: repartition increases or changes partitions and performs a full shuffle.

Q12. What is coalesce()?

Answer: coalesce reduces partitions with less shuffle and is commonly used before writing output to reduce tiny files.

Q13. What is broadcast join?

Answer: Broadcast join sends a small table to all executors so a large-small join can happen with less shuffle.

Q14. What is Spark SQL?

Answer: Spark SQL is Spark's module for DataFrame and SQL processing.

Q15. What is Structured Streaming?

Answer: Structured Streaming is Spark's streaming engine built on DataFrame-like APIs.

Q16. How do you debug a slow Spark job?

Answer: Use Spark UI to inspect jobs, stages, tasks, shuffle, skew, executor memory and SQL physical plans.

15. Quick Revision Sheet

Remember	Meaning
SparkSession	Main entry point for Spark application.
RDD	Low-level resilient distributed dataset.
DataFrame	Distributed table with schema and optimizer support.
Transformation	Creates a new logical dataset; lazy.
Action	Triggers job execution.
DAG	Graph of execution dependencies.
Stage	Group of tasks separated by shuffle boundaries.
Task	Small unit of work on a partition.

Remember	Meaning
Shuffle	Network movement of data; expensive.
Cache	Stores reused data in memory/disk to avoid recomputation.
Final trainer tip For interviews, explain Spark using this order: Driver -> Cluster Manager -> Executors -> Partitions -> Transformations -> DAG -> Stages -> Tasks -> Action.	